

An Exploration of Dynamic Invariant Generation with Daikon

Vishwesh Palani
vpalani@umass.edu

Introduction

The focus of this independent study revolved around exploring the capabilities of *Daikon* (Ernst et al.), an open-source tool that can generate program invariants for computer programs by analyzing execution traces when evaluated with a complete test suite. While Daikon can produce various types of invariants, this study focused on loop invariant generation for basic search and arithmetic operations. The applications of Daikon invariants with respect to Automated Program Repair were briefly explored as well.

Literature Review

Two primary classes of invariant generation have been seen in much investigation: static and dynamic analysis. Static analysis involves directly observing the code to infer likely invariants through constraint-solving and forward-propagation techniques including but not limited to symbolic execution. This method of invariant generation is generally much harder to perform successfully, yet it has strong applications in program verification – when supplied with invariants, verifiers like Z3 Solver, Dafny, etc. can ensure that the invariants hold throughout the program (Nimmer and Ernst). Dynamic analysis, on the other hand, involves running programs to infer state space properties by analysing execution traces. It is a relatively simpler technique but is largely error-prone, often inferring invariants that are, true but, irrelevant to the program specification (Al-Bataineh). Daikon is of the latter variety and has been widely successful given a proper test suite.

Methodology

This study evaluates Daikon’s ability to generate loop invariants for simple programs. A suite of programs modelling search algorithms and basic arithmetic operations will be assembled as in Abreu et al. Next, a test suite will be created for each program. Each test suite will be run first with the *DynComp* comparability analysis tool that performs dynamic type inference and groups program variables by type into comparability sets. This step prevents Daikon from generating meaningless invariants between variables of entirely different types. Next, the *Chicory* front end is used to generate the execution trace which will then be run through Daikon to generate the invariants.

The final program suite consisted of 16 distinct Java programs. Some functions simply wrapped the Java arithmetic operators (for multiplication, integer division, addition and subtraction) as well as other iterative implementations of taking the absolute value, exponentiation, and performing the Euclidean algorithm. Recursive implementations of these mathematical operations were included as well to observe any changes to the invariants (the test suites remained the same across all variations). A *SumArray* function (which calculated the sum of the elements in a number array) and several

implementations of Binary Search (recursive and iterative) were tested as well to see the dynamically generated invariants for array functions.

Results and Analysis

All invariants consistently verified the types and ranges of inputs that were accepted by the functions, although this was based on the inputs supplied by the test suite rather than the program specification. For arithmetic operations, input values were compared with return values and each other through equality checks (invariants sometimes in the form of $x \neq y$). All notable invariants have been briefly detailed in the table below.

Function	Implementation	Notable Invariants
Addition	Operator	$return - orig(x) - orig(y) == 0$
	Recursion	Empty Invariant
Multiplication	Operator	$return \% orig(x) == 0, return \% orig(y) == 0$
	Recursion	$return \% orig(x) == 0$
Integer Division	Operator	Empty Invariant
Remainder	Operator	
Exponentiation	Iteration	$base, exp, return \geq 0$
Euclidean Algorithm	Iteration	$orig(a / b) \% return == 0, 1 \leq return \leq orig(a / b)$
	Recursion	Fig. 1
Absolute Value	Function	$orig(x) \% return == 0, return \% orig(x) == 0, return \geq orig(x)$
	Ternary If/Else	Identical to above
Array Summation	Iteration	Fig. 2
Binary Search	Iteration	Fig. 3
	Recursion	

Some, rather, complicated implementations that depended on a long series of seemingly unrelated, minute, computations (such as the recursive implementation of the addition operator and integer division)

```
=====
GCD.gcd(int, int)::ENTER
a >= 0
b >= 0
=====
GCD.gcd(int, int)::EXIT6
return == orig(b)
orig(a) == 0
return > orig(a)
=====
GCD.gcd(int, int)::EXIT7
orig(a) >= 1
return <= orig(a)
=====
GCD.gcd(int, int)::EXIT
(orig(a) == 0) <=> (return > orig(a))
(orig(a) == 0) ==> (orig(a) < orig(b))
(orig(a) == 0) ==> (orig(b) >= 1)
(orig(a) == 0) ==> (return == orig(b))
return >= 1
orig(a) % return == 0
orig(b) % return == 0
=====
```

Fig. 1: Recursive Euclidean Algorithm

resulted in empty invariants. The most basic functions produced the most detailed invariants that linked the return values and inputs in a relevant manner (e.g. Java multiplication/addition operator and both absolute value implementations). The more complicated the functions became, the harder it became for Daikon to infer invariants that accurately described the program specification using only comparison operators.

```
=====
SumArrayOp.sum(int[])::ENTER
arr != null
arr[] elements >= 0
size(arr[])-1 != 0
=====
SumArrayOp.sum(int[])::EXIT
arr[] == orig(arr[])
arr[] elements >= 0
size(arr[])-1 != 0
arr[] elements < return
=====
```

Fig. 2: Array Summation

A noteworthy observation is that recursive implementations produced a drastically larger number of invariants as opposed to their iterative counterparts as demonstrated by the Euclidean Algorithm and Binary Search implementations (among others). This is primarily because Daikon records the values of input parameters and return values only. Local variables are not recorded so every function call mandates additional data to be stored and analyzed. To take advantage of this, dummy procedures were called, as recommended by the Daikon Documentation, wherein desired global and local variables were passed into an empty Java method so that the values would be stored. In this suite of programs, there were no significant changes to the invariants but additional relationships were captured.

Despite the difference in quantity between recursive and iterative implementations, the invariants for recursion yielded no useful information about the program specification, rather it yielded information about the test suite and its limitations. For example, if Binary Search was only tested with a target value of 5894 as can be seen in Fig. 4, that would be identified as an invariant.

Fig. 4: Binary Search - Iteration (Left), Recursion (Right)

```

BinarySearchOp.search(int[], int)::ENTER
arr != null
arr[] elements == 0
arr[] sorted by <=
x one of { 7, 5894 }
size(arr)-1 == 0
=====
BinarySearchOp.search(int[], int)::EXIT11
orig(x) == arr[return]
orig(x) == orig(arr[post(return)])
orig(x) == 5894
return == size(arr)-1
=====
BinarySearchOp.search(int[], int)::EXIT24
arr[] elements one of { 1, 3, 9 }
arr[] one of { [], [1, 3, 9] }
return == -1
orig(x) == 7
orig(size(arr)) one of { 0, 3 }
=====
BinarySearchOp.search(int[], int)::EXIT
arr[] == orig(arr[])
return == -1 ==> (orig(x) == 7)
return == -1 ==> (arr[] elements one of { 1, 3, 9 })
return == -1 ==> (arr[] one of { [], [1, 3, 9] })
return == -1 ==> (orig(size(arr)) one of { 0, 3 })
arr[] elements == 0
arr[] sorted by <=
return == -1
size(arr)-1 == 0
return == size(arr)-1
=====

BinarySearchOp::OBJ167
=====
BinarySearchOp.BinarySearchOp::EXIT
BinarySearchOp.loop_dummy(int, int, int, int)::ENTER
l == 0
x one of { 7, 5894 }
l <= r
l < arr.length
r < arr.length
=====
BinarySearchOp.loop_dummy(int, int, int, int)::EXIT
=====
BinarySearchOp.search(int[], int)::ENTER
arr != null
arr[] elements == 0
arr[] sorted by <=
x one of { 7, 5894 }
size(arr)-1 == 0
=====
BinarySearchOp.search(int[], int)::EXIT15
orig(x) == arr[return]
orig(x) == orig(arr[post(return)])
orig(x) == 5894
orig(x) == arr[return-1]
=====
BinarySearchOp.search(int[], int)::EXIT26
arr[] elements one of { 1, 3, 9 }
arr[] one of { [], [1, 3, 9] }
return == -1
orig(x) == 7
orig(size(arr)) one of { 0, 3 }
=====
BinarySearchOp.search(int[], int)::EXIT
arr[] == orig(arr[])
return == -1 ==> (orig(size(arr)) one of { 0, 3 })
return == -1 ==> (orig(x) == 7)
return == -1 ==> (arr[] elements one of { 1, 3, 9 })
return == -1 ==> (arr[] one of { [], [1, 3, 9] })
arr[] elements == 0
arr[] sorted by <=
return == 0
size(arr)-1 == 0
return == size(arr)-1
=====
BinarySearchTest.generateRandomArray::EXIT
return != null
return[] elements == 0
=====
BinarySearchTest.insertAndSort::EXIT
return != null
return[] elements == 0
return[] sorted by <=
=====
BinarySearchTest.main(java.lang.String[])::ENTER
args has only one value
args.getClass().getName() == java.lang.String[].class
args[] == []
args[].toString == []
=====
BinarySearchTest.main(java.lang.String[])::EXIT
args[] == orig(args[])
args[] == []
args[].toString == []
Exiting Daikon.

```

Unfortunately, due to the absence of a proper template, Daikon was unable to infer polynomial invariants and relationships between array elements with its default set of templates.

Evaluation

The invariants revealed any flaws in the test suites, suggesting that the quality of the test suites was sufficient to generate invariants, but not enough to infer relatively useful ones. Moreover, the usage

of dummy method calls was only applied to Binary Search, and while it yielded nothing useful, that may have not been the case for other programs. Moreover, several other implementations of these programs and others could have been tested for a more holistic analysis of Daikon's capabilities.

Conclusion

An unexpected takeaway from this study is that Daikon, and dynamic inference in general, may be used to estimate the quality of a given test suite. One possible strategy is to keep improving test suite quality until the generated invariants no longer change (i.e. converge). Daikon is very good at drawing comparisons between its inputs and return values but cannot detect loop and array invariants independently mandating the use of templates. Further areas of research include the development of templates to aid polynomial invariant generation and exploring the applications of dynamically inferred invariants concerning automated program repair (Goues et al.).

References

- Abreu, Alexandre, et al. "Exploring Automatic Specification Repair in Dafny Programs (Ase 2023 - [Workshop] Asyde) - ASE 2023." *Conf.Researchr.Org Conference Management System* -, conf.researchr.org/details/ase-2023/ase-2023--workshop--asyde/8/Exploring-Automatic-Specificati-on-Repair-in-Dafny-Programs. Accessed 17 May 2024.
- Al-Bataineh, Omar I. "Invariant-Based Program Repair." *arXiv.Org*, 26 Jan. 2024, arxiv.org/abs/2312.16652.
- Ernst, Michael D., et al. "The Daikon System for dynamic detection of likely invariants." *Science of Computer Programming*, vol. 69, no. 1–3, Dec. 2007, pp. 35–45, <https://doi.org/10.1016/j.scico.2007.01.015>.
- Goues, Claire Le, et al. "Automated Program Repair." *Communications of the ACM*, vol. 62, no. 12, 21 Nov. 2019, pp. 56–65, <https://doi.org/10.1145/3318162>.
- Nimmer, Jeremy W., and Michael D. Ernst. "Static verification of dynamically detected program invariants." *Electronic Notes in Theoretical Computer Science*, vol. 55, no. 2, Oct. 2001, pp. 255–276, [https://doi.org/10.1016/s1571-0661\(04\)00256-7](https://doi.org/10.1016/s1571-0661(04)00256-7).